

\). In replacement text, \3 is the third sub-string/group found; \2 the second, and \1 the first match.

Quoting

For Sed scripts, we can use single quotes or double quotes. When single quotes are used (*sed 'STATEMENTS' file*), it will not expand shell variables. But with double quotes (*Sed "STATEMENTS" file*) Bash will expand shell variables before passing the arguments to Sed. Therefore, we should use double quoting if we need to pass values into the Sed script from Bash. For example:

```
$ replace_txt="REPLACED"
$ match="MATCH"
$ echo This is a MATCH text | sed "s/$match/$replace_txt/"
This is a REPLACED text
```

Combining multiple Sed statements

Combining multiple Sed commands using a pipe (*sed 'expression' | sed 'expression'*) can be replaced with *sed 'expression; expression'*.

Restrict range of affected lines with patterns and line numbers

We can specify the range of line numbers for which text manipulation operations are to be performed. It is also possible to specify a range of lines by specifying a start-line pattern and an end-line pattern. To specify a range of line numbers, use something like what's shown below:

```
$ seq 30 | sed '10,20 s/[0-9]$/''
```

In the output, you can see that in lines 10-20, the last digit has been removed (replaced with an empty string). You can specify the ending line number to be the last line in the file by using a \$ character as follows (10,\$):

```
$ seq 30 | sed '10,$ s/[0-9]$/''
```

Now, let us look at how to specify ranges of lines by using patterns. Let's suppose that I need to apply a text operation to a range of text that is between one line with the word 'start', and another with the word 'end', in a sample file (call it eg.txt) with these contents:

```
hello
next
start
line1
line2
end
next
```

We run this command, and observe in the output that only the desired lines have been modified:

```
$ sed '/start/, /end/ s/./& modified/' eg.txt
hello
next
a line with start  modified
line1 modified
line2 modified
a line with end modified
next
```

Here, */START_PAT/, /END PAT/* is used to identify the text range to be operated on. You can mix line numbers and text matches while specifying ranges; */START_PAT/, \$* will operate on text between *START_PAT* and the end of file, while *1, /END_PAT/* will operate from the first line to a line containing *END_PAT*.

Grouping commands into subgroups

Grouping Sed statements according to contexts is a very useful operation. We can perform operations by matching some pattern of the input, then apply another text operation after the pattern is matched, after which we can again operate on a smaller subgroup, and so on. For example, here is a problem: in the file *eg2.txt* (contents listed below) I may change *only* text between the 3rd and 8th lines. In that range, I must operate on (the range of) all lines between the patterns 'single' and 'double'. In this nested range, I must replace all the lines' text with #:

```
eg2.txt:
first line
2nd line
3rd line
a word single
next line
next lines
double
test line
last line
```

After creating the text file, we run the following command on it and observe that the output satisfies the constraints and does the required operation:

```
$ sed '3,8 { /single/,/double/ { s/./#/ } }' eg2.txt
first line
2nd line
3rd line
#
#
#
#
test line
last line
```